AI Tax Advisor

Intelligent Tax Guidance Web Application

Full-stack web application powered by AI that provides personalized tax guidance through structured form input and AI-generated guidance. Built with React, FastAPI, OpenAI, and modern DevOps practices.

Developed by Arsenis Chrysikopoulos

Project Overview

AI-Powered Guidance

Generates personalized tax advice using OpenAI's language models

Web Interface

React-based frontend for collecting user financial data

Backend Processing

FastAPI handles data validation and AI integration

Users input their financial and tax-related information through a structured form. The backend processes this data, constructs a contextual prompt, and retrieves AI-generated guidance that's displayed in a readable format.

Disclaimer: The app provides general educational guidance, not certified tax or legal advice.

Problem & Solution

The Problem

- Tax systems are notoriously complex and difficult to navigate
- Professional advice is expensive and not always accessible
- Generic resources don't address individual circumstances

Our Solution

- AI-powered assistant that translates complex inputs into clear guidance
- Personalized advice based on user's specific financial situation
- Accessible guidance to help users better understand their situation

How It Works

User Input

Tax form with financial data

API Request

Frontend sends validated data

AI Processing

Backend builds prompt and queries OpenAI

Guidance Display

Markdown-formatted advice rendered in UI

System Architecture

1

React Frontend

Interactive UI with form validation and responsive design

2

FastAPI Backend

REST API handling business logic and data processing

3

OpenAI Integration

AI-powered tax guidance generation

4

Docker & CI/CD

Containerized deployment with automated testing

Frontend Implementation

Form Validation

React Hook Form combined with Zod for robust type-safe validation

Type Safety

TypeScript ensures consistent data structures across components

Responsive Design

Accessible interface that works across all device sizes

Markdown Rendering

AI responses formatted for clear, readable presentation

Backend & AI Integration

FastAPI Features

- RESTful endpoints with clear routing
- Pydantic models for data validation
- Modular structure separating concerns

AI Integration

- Server-side API calls keep keys secure
- Structured prompts ensure consistent output
- Service layer isolates AI logic

Security First: API keys never exposed to frontend. All AI requests processed through secure backend endpoints.

Key Engineering Decisions

Stateless Design

No persistent database—session data stored client-side for simplicity

Separation of Concerns

Clear boundaries between UI, API, and AI service layers

AI Isolation

Service layer abstracts OpenAI calls for easier testing and maintenance

Minimal Architecture

Local state management over complex global stores

Challenges & Solutions

1

Prompt Design

Challenge: Getting useful, consistent AI responses

Solution: Structured, contextual prompts with clear formatting instructions

2

Output Consistency

Challenge: Variable AI response formats

Solution: Markdown output with sanitization for security

3

Security

Challenge: Protecting API credentials

Solution: Backend-only API calls with environment variables

4

Architecture Simplicity

Challenge: Avoiding overengineering

Solution: State management tailored to app complexity

Future Improvements

User Authentication

Add login system for personalized experience

Data Persistence

Store user profiles and history securely

Structured Output

Improved AI response formatting with JSON

Cloud Deployment

Production-ready infrastructure on AWS/Azure

Project demonstrates: Full-stack development with React and FastAPI • AI integration best practices • Modern DevOps with Docker and CI/CD • Security-conscious architecture • User-centered design